PLAYWRIGHT PROJECT

ONLINE SHOPPING AUTOMATION

JavaScript | Page Object Model | UI Testing

Customer workflows, framework design and execution

Project Information

OVERVIEW

Customer journeys from sign-in to order tracking

11
Business scenarios

24
Page object files

4
Product categories

Application scope

Products, cart, wishlist, checkout, reviews, account details and order tracking.

Current browser

Chromium / Desktop Chrome

One additional setup test creates reusable authentication state.

Project Business Need

PURPOSE

Protect shopping journeys

Check that key customer tasks continue to work after application changes.

Reduce repeated manual work

Repeat stable regression scenarios with consistent steps and expected results.

Verify saved state and totals

Check cart arithmetic, saved details, wishlist persistence and session behavior.

Make failures explainable

Use named tests, HTML results and available screenshots or traces as evidence.

Application Modules

BUSINESS ARCHITECTURE

ONLINE SHOPPING APPLICATION

Discover products

Search
Books and Fashion
Furniture and Electronics

Shop and purchase

Cart and Wishlist
Checkout and Pending
Payments
Order Tracking

Manage the account

Login and Logout
Profile, Billing and Password
Product Reviews

The test suite connects these modules through customer-facing workflows.

Framework Architecture

AUTOMATION DESIGN



Inputs and setup

JSON test data
Unique shopper fixture
Saved authentication state

Responsibilities

Specs describe scenarios and assertions.
Page objects expose locators and actions.
Checks validate the observed UI behavior.

Assertions → Test results → HTML report + configured artifacts

Technology & Structure

FRAMEWORK ORGANIZATION

Technology stack

- JavaScript and Node.js
- Playwright Test and Chromium
- Page Object Model and JSON data
- Jenkins and GitHub Actions
- HTML reports, screenshots and traces

Project organization

```
tests/                  Scenarios + support
POM/                    Page objects
testData/               Reusable inputs
playwright.config.js    Execution setup
Jenkinsfile             CI pipeline
screenshots/            Tracking evidence
```

Supporting material: Manual_Test_Scripts.xlsx and the framework presentation guide.

Login & Logout

MODULE 01 / ACCOUNT SESSION

Validate account identity and signed-out behavior

Register + sign in → **Verify account** → **Sign out + revisit**

Before logout

Welcome message matches the shopper.
Account email matches the created user.

After logout

Login link returns; logout link disappears.
Account email stays empty after reload.

Key files: loginLogout.spec.js • loginLogout.page.js • shopping.fixture.js

Product Search & Cart

MODULES 02–03 / PRODUCT DISCOVERY AND CALCULATIONS

Search the correct product

Enter the configured keyword.
Open the exact matching product.
Verify its details heading.

Add the same product repeatedly

Add the product three times.
Verify one row; quantity grows from 1 to 3.
Check price, shipping and totals.

Row total = quantity × unit price + shipping

For the tested single-product cart: grand total must equal the row total.

Wishlist Persistence

MODULE 04 / SAVED PRODUCTS

A saved product should survive a new login

01 → **02** → **03** → **04**

Open product

Verify the product heading

Save to wishlist

Verify one saved row

Logout + login

Use the same account

Reopen wishlist

Verify the saved product

Persistence check: saved data remains available across the session boundary.

Wishlist to Cart

MODULE 05 / SAVED-PRODUCT PURCHASE INTENT

Save product → **Read wishlist price** → **Add to cart**

Price and quantity

Cart price equals the saved-product price.
One cart row contains one unit.
Row and grand totals are consistent.

Persistence and identity

Reload the page and check the cart again.
Open the product using its cart link.
Verify the expected product heading.

Reusable assertion helper: checkCart(cart, 1)

Product Reviews

MODULE 06 / PRODUCT-SPECIFIC PERSISTENCE

Create a distinct review

Open the intended product.
Select the configured ratings.
Enter a unique summary and review text.

Verify it on the same product

Revisit the product and open reviews.
Find exactly one matching summary.
Check reviewer name and review content.

Unique review identifier → **Revisit product** → **Assert persisted content**

The checks validate saved UI content; submitted reviews currently remain in the application.

Billing & Profile Updates

MODULES 07–08 / CUSTOMER DETAILS

Billing address

Update the configured address fields.
Save and reload the account page.
Compare persisted field values.

Starting state: saved authentication.

Profile details

Update the shopper name and contact.
Check after reload and re-login.
Restore the original name and contact.

Starting state: unique shopper fixture.

The key assertion is persistence after a refresh or a new login session.

Password Change Validation

MODULE 09 / NEGATIVE ACCOUNT SCENARIO

Reject an incorrect current password

- Open the change-password form for the signed-in shopper.
- Enter an incorrect current password and a proposed new password.
- Verify the expected rejection message.
- Sign in again using the original password.

Expected behavior: the invalid request leaves the existing password usable.

Checkout & Pending Payments

MODULE 10 / INCOMPLETE CHECKOUT

Add a product → **Reach checkout** → **Leave payment pending**

Business check

A shopper can find an unpaid product in the pending-payments view.

Implemented assertion

The pending-payments page contains the expected product name.

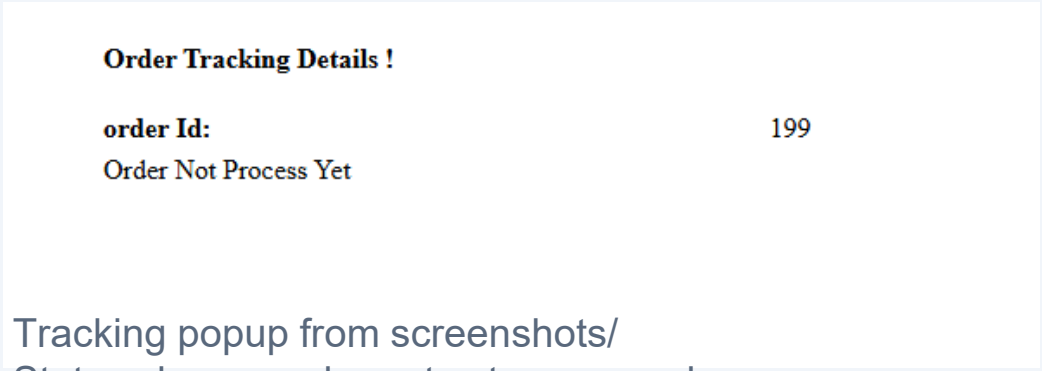
Next improvement: correlate the assertion to the exact order created by this test.

Purchase & Order Tracking

MODULE 11 / END-TO-END JOURNEY

- Register and sign in with a new account.
- Add products from all four categories.
- Set billing and shipping, then check out.
- Submit the internet-banking option.
- Verify links and capture each popup.

Existing project evidence



Tracking popup from screenshots/
Status shown: order not yet processed

Coverage: UI checkout and tracking availability; payment settlement is not asserted.

Functional Dependencies

AUTHENTICATION AND TEST SETUP

Unique shopper fixture

Fresh shopper per test.
Six scenarios use the fixture.
Session teardown follows.

Shared authentication

Setup saves the login state.
Two specs reuse the session.
Setup runs before both specs.

Scenario-specific setup

Three other scenarios arrange their own starting state.
JSON supplies reusable input values.

setup → **Saved browser state** → **chromium-authenticated**

Execution uses one worker to reduce conflicts with shared account data.

Business Workflows

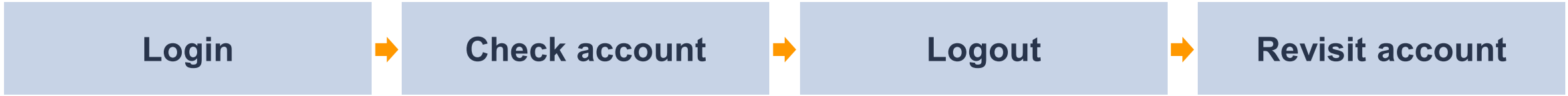
FROM USER ACTION TO ASSERTION

Workflow A / Saved product to purchase intent



Assertions: product identity • price equality • quantity • totals • persisted cart

Workflow B / Authenticated user to signed-out state



Assertions: welcome and email match → login link returns → account email stays empty.

Execution & Reporting

RUNNING AND INVESTIGATING TESTS

PowerShell commands

```
npx playwright test

npx playwright test `
  --project=chromium-authenticated

npx playwright show-report
```

Execution settings

- Chromium; one worker
- Two retries when CI is enabled
- HTML results for each test run
- Failure artifacts where configured

Prerequisite: configure BASE_URL. Coverage shown here is based on source, not a new test run.

CI & Delivery Workflow

CONTINUOUS INTEGRATION

Jenkins

Windows commands (bat)
BASE_URL from a Jenkins credential
Install dependencies and browsers
Run tests and publish the HTML report

GitHub Actions

Push / pull request to main or master
Ubuntu runner with Node.js
Install, run tests and upload the report
Report artifact retention: 30 days

Install dependencies → **Run Playwright tests** → **Publish report**

Configuration gap: supply BASE_URL in GitHub Actions before running the workflow.

Coverage & Next Improvements

IMPLEMENTED SCOPE AND DEVELOPMENT PRIORITIES

Implemented scenarios

Shopping and discovery	**5**
Account management	**4**
Checkout and orders	**2**
Total business scenarios	**11**

Next improvements

- Repair both wishlist cleanup hooks.
- Verify exact orders and stronger checkout outcomes.
- Complete account / review data cleanup.
- Configure CI inputs and expand browser coverage.

The framework provides reusable workflow coverage and a clear path to stronger evidence.

THANK YOU

Questions & Discussion

11 scenarios. Reusable page objects.
Customer-focused assertions.